VSEPR-SIM — Graphing Module Plan

Python Visualisation Suite for Atomistic Simulation Data

Planning Document v1.0 — April 2026

VSEPR-SIM V4 Research Platform

Contents

This document describes the planned Python graphing module suite. No implementation code is written here. All modules consume JSON snapshots produced by `crystal_card_popup.py` and future simulation output files. Code will be written in a separate phase after this design is reviewed.

§G0 — Design Philosophy

The graphing suite follows the same anti-black-box principle as the rest of VSEPR-SIM: every curve, every data point, and every axis must be directly traceable to a specific simulation output. No synthetic or interpolated data should be presented without explicit labelling.

Three core ideas govern the design:

1. **Data loads visibly.** Curves animate from left to right as data arrives. The user sees the process, not just the result.
2. **2D and 3D are co-equal.** Every dataset has a designated 2D representation (publication-ready) and a designated 3D representation (exploratory, rotatable).
3. **Modules are single-purpose.** Each module reads exactly one data type, produces exactly one figure class. No omnibus dashboards at this stage.

§G1 — Data Sources

G1.1 Card Snapshot JSON

Produced by `crystal_card_popup.py` on every card open. Location: `data/card_snapshots/{symbol}_{lattic`

Fields available for graphing:

Field	Type	Use
timestamp_ms	int	timeline axis
symbol	str	per-element colour coding
lattice	str	group/facet key
a0_angstrom	float	EOS / lattice parameter plot
Tmelt_K	float	thermal curve overlay
Ecoh_eV	float	cohesive energy chart
gamma	float	score distribution / radar
q_data	float	score distribution / radar
compact	float	score distribution / radar
render_mode	str	2D vs. 3D session split
complexity_label	str	supercell tier histogram
supercell_dims	list	atom-count scaling curve
view_azimuth_deg	float	viewing angle distribution
view_elevation_deg	float	viewing angle distribution

G1.2 Future Simulation Output Files

Planned formats (not yet implemented):

- data/eos/{symbol}_eos.csv — columns: volume_AA3, energy_eV, pressure_GPa
- data/rdf/{symbol}_rdf.csv — columns: r_AA, g_r
- data/trajectory/{symbol}_md.xyz — XYZA format; existing reader in src/io/xyz_format.cpp
- data/convergence/{symbol}_conv.csv — columns: step, energy_eV, force_max_eVAA
- data/thermo/{symbol}_thermo.csv — columns: step, T_K, P_GPa, E_total_eV
- data/phonon/{symbol}_dos.csv — columns: freq_THz, dos

§G2 — Planned Module List

Module file	Type	Data source	Description
`plot_score_dist.py`	2D	snapshots	Histogram + KDE of γ, Q, C across all recorded sessions
`plot_score_radar.py`	2D	snapshots	Animated radar polygon building up per-element as snapshots are scanned
`plot_eos.py`	2D	eos CSV	Energy–volume curves with Birch–Murnaghan fit overlay
`plot_rdf.py`	2D	rdf CSV	$g(r)$ with animated trace revealing from $r = 0$
`plot_rdf_3d.py`	3D	rdf CSV	$g(r, \theta)$ surface in Matplotlib 3D axes
`plot_convergence.py`	2D	conv CSV	Energy and force vs. step; dual-axis; animated left-to-right
`plot_thermo.py`	2D	thermo CSV	T, P, E vs. MD step; triple panel; animated reveal
`plot_thermo_3d.py`	3D	thermo CSV	T–P–E phase-space trajectory; points appear sequentially
`plot_trajectory_2d.py`	2D	XYZA	Per-atom displacement vs. step as translucent line bundle
`plot_trajectory_3d.py`	3D	XYZA	3D scatter of atom positions; animated by frame
`plot_lattice_param.py`	2D	snapshots	a_0 vs. T_{melt} scatter, colour-coded by lattice type
`plot_ecoh_bar.py`	2D	snapshots	E_{coh} bar chart; bars grow left-to-right
`plot_phonon_dos.py`	2D	phonon CSV	Phonon DOS; shaded area fills in from low to high frequency
`plot_session_timeline.py`	2D	snapshots	Dot-strip timeline coloured by element; renders live
`plot_supercell_scale.py`	2D	snapshots	Atom count vs. supercell tier; FCC/BCC/HCP side by side

§G3 — Animated Data-Loading Pattern

Every module that touches time-series or step-indexed data *must* implement the animated loading pattern. The intended visual effect: the curve or scatter plot is empty at $t = 0$; data points appear one-by-one (or in small batches) from left to right, building the full picture in under 3 seconds.

G3.1 Planned mechanism

- Load the full data array into memory first (no streaming required).
- Use Matplotlib `FuncAnimation` with a frame count equal to the number of data points (or a fixed 60–120 frames with sub-sampling).
- Each frame reveals the next batch of k points ($k = \lceil N/90 \rceil$ for approximately 1.5 s at 60 fps).
- For line plots: update `line.set_data(x[:frame], y[:frame])`.

- For scatter plots: replace the `PathCollection` offsets each frame.
- For area/fill plots: redraw `fill_between` on a clipped slice.

G3.2 Axes style

All modules share a common dark style consistent with the card UI:

Element	Value
Figure background	`#0d0d1a`
Axes background	`#111130`
Tick / label colour	`#888888`
Grid colour	`#222244`, dashed
Accent colour	`#00d4ff`
Secondary colour	`#44ffaa`
Warning colour	`#ffaa44`
Font	Consolas or DejaVu Sans Mono

G3.3 Export

Every module saves the completed figure to `data/figures/{module_stem}_{timestamp}.png` at 150 dpi by default, 300 dpi with `--hires` flag. Animated versions save `.gif` alongside.

§G4 — 3D Module Specifics

3D modules use `matplotlib.pyplot` with `projection='3d'`. No external OpenGL or VTK dependency is introduced at this stage.

G4.1 Rotation

All 3D plots are initialised with: `ax.view_init(elev=22, azim=30)` — the same hardcoded angles used by the crystal card viewer.

G4.2 Depth cues

- Scatter point size scales with depth (nearer = larger).
- Alpha channel encodes depth (nearer = more opaque).
- Line bundles use `LineCollection` with per-segment alpha.

G4.3 Planned 3D figures and axes

Module	X axis	Y axis	Z axis
`plot_rdf_3d.py`	r (Å)	θ (rad)	$g(r, \theta)$
`plot_thermo_3d.py`	T (K)	P (GPa)	E (eV)
`plot_trajectory_3d.py`	x (Å)	y (Å)	z (Å)

§G5 — Module Directory Layout

```
tools/
  graphing/
    __init__.py                    (empty, marks as package)
```

```
    _style.py                    (shared dark style + colour palette)
    _loader.py                   (JSON snapshot reader + CSV reader helpers)
    _anim.py                     (FuncAnimation wrapper + export helper)
    plot_score_dist.py
    plot_score_radar.py
    plot_eos.py
    plot_rdf.py
    plot_rdf_3d.py
    plot_convergence.py
    plot_thermo.py
    plot_thermo_3d.py
    plot_trajectory_2d.py
    plot_trajectory_3d.py
    plot_lattice_param.py
    plot_ecoh_bar.py
    plot_phonon_dos.py
    plot_session_timeline.py
    plot_supercell_scale.py

data/
  card_snapshots/    (JSON -- written by crystal_card_popup.py)
  eos/               (CSV  -- future EOS engine)
  rdf/               (CSV  -- future RDF engine)
  convergence/       (CSV  -- future SCF/MD engine)
  thermo/            (CSV  -- future MD thermostat)
  trajectory/        (XYZA -- future MD engine)
  phonon/            (CSV  -- future phonon engine)
  figures/           (PNG + GIF output from graphing modules)
```

§G6 — CLI Interface (Planned)

Each module is callable as a standalone script:

```
python tools/graphing/plot_score_dist.py [--snapshot-dir PATH] [--hires]
python tools/graphing/plot_eos.py Au Fe Ti [--overlay] [--hires]
python tools/graphing/plot_convergence.py Au [--no-anim]
python tools/graphing/plot_trajectory_3d.py Au [--frame N]
```

All modules accept `--no-anim` to skip animation and render the final state immediately (for batch or CI use).

§G7 — Dependencies (Minimal)

Library	Version	Use
`matplotlib`	≥ 3.7	all 2D and 3D plots, animation
`numpy`	≥ 1.24	array operations, FFT (phonon DOS)
`scipy`	optional	Birch–Murnaghan fit in `plot_eos.py`
`json`	stdlib	snapshot loading
`csv`	stdlib	CSV loading

No Plotly, Bokeh, VTK, or heavy GUI frameworks. The suite is intentionally minimal and portable.

§G8 — Implementation Order

Modules are prioritised by data availability and scientific utility:

Phase	Module(s)	Blocker
1	`plot_score_dist`, `plot_score_radar`	snapshots already produced
1	`plot_lattice_param`, `plot_ecoh_bar`, `plot_supercell_scale`	snapshots already produced
1	`plot_session_timeline`	snapshots already produced
2	`plot_eos`	EOS CSV engine
2	`plot_convergence`	SCF/MD convergence writer
3	`plot_rdf`, `plot_rdf_3d`	RDF engine
3	`plot_thermo`, `plot_thermo_3d`	MD thermostat writer
4	`plot_trajectory_2d`, `plot_trajectory_3d`	XYZA MD trajectory writer
4	`plot_phonon_dos`	phonon engine

Phase 1 modules can be written immediately against live snapshot data. All Phase 1 modules are 2D; 3D modules appear from Phase 3 onward.

§G9 — Notes and Constraints

- No atomistic-level simulation data is invented or hardcoded. If a data file does not exist, the module exits with a clear message.
- Terminology: use *atomistic* throughout. The word *meso* is forbidden in all module code, comments, and axis labels.
- All modules must run without a display server when `--no-anim` is passed (`matplotlib.use('Agg')` fallback).
- Figures are research-output quality: axis labels carry units, titles carry element symbol and lattice type.
- The `_style.py` shared module is the single source of truth for colours and font settings. Individual modules must not hardcode style constants.
